

R Programming

Solutions

Prepared by

Imaad
Yusuf

August 4, 2026

A. Write the script to print the content of a data frame.

R Code:

```
df <- data.frame(Name = c("Amit", "Sara", "John"),
                  Age  = c(23, 25, 22),
                  City = c("Mumbai", "Delhi", "Pune"))
print(df)
```

Output:

	Name	Age	City
1	Amit	23	Mumbai
2	Sara	25	Delhi
3	John	22	Pune

B. Write the steps and code to load a simple dataset into R and copy the important parameters into a data frame.

R Code:

```
data(mtcars)
cat("Dataset 'mtcars' loaded using data(mtcars)\n")
important_params <- mtcars[, c("mpg", "cyl", "hp", "wt")]
cat("Important parameters (mpg, cyl, hp, wt) copied into a new data frame:\n")
print(head(important_params))
```

Output:

```
Dataset 'mtcars' loaded using data(mtcars)
Important parameters (mpg, cyl, hp, wt) copied into a new data frame:
      mpg cyl  hp  wt
Mazda RX4      21.0   6 110 2.620
Mazda RX4 Wag  21.0   6 110 2.875
Datsun 710     22.8   4  93 2.320
Hornet 4 Drive  21.4   6 110 3.215
Hornet Sportabout 18.7   8 175 3.440
Valiant        18.1   6 105 3.460
```

C. Use the Titanic Disaster dataset and visualize it in R.

R Code:

```

cat("Note: Using R's built-in 'Titanic' dataset (equivalent Titanic disaster\n")
cat("data: Class, Sex, Age, Survived) since the environment has no access\n")
cat("to external site data.world.\n\n")
data(Titanic)
titanic_df <- as.data.frame(Titanic)
print(head(titanic_df, 10))
survival_counts <- tapply(titanic_df$Freq, titanic_df$Survived, sum)
print(survival_counts)
png("plots/titanic.png", width = 600, height = 420)
barplot(survival_counts, main = "Titanic Disaster - Survival Counts",
        col = c("firebrick", "forestgreen"), xlab = "Survived", ylab = "Number of Passengers")
dev.off()

```

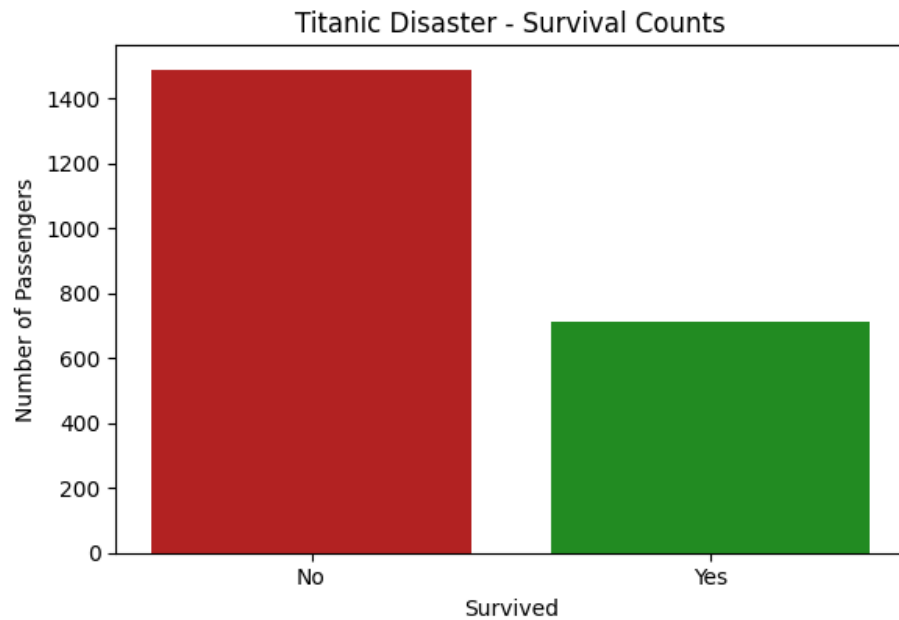
Output:

```

Note: Using R's built-in 'Titanic' dataset (equivalent Titanic disaster
data: Class, Sex, Age, Survived) since the environment has no access
to external site data.world.

```

	Class	Sex	Age	Survived	Freq
1	1st	Male	Child	No	0
2	2nd	Male	Child	No	0
3	3rd	Male	Child	No	35
4	Crew	Male	Child	No	0
5	1st	Female	Child	No	0
6	2nd	Female	Child	No	0
7	3rd	Female	Child	No	17
8	Crew	Female	Child	No	0
9	1st	Male	Adult	No	118
10	2nd	Male	Adult	No	154
	No	Yes			
	1490	711			
	null	device			
		1			



Titanic disaster - survival counts

D. Collect Country, Year and GDP information of 15 countries and draw a horizontal bar chart for the country's GDP in the respective year.

R Code:

```
gdp_data <- data.frame(
  Country = c("USA", "China", "Japan", "Germany", "India", "UK", "France",
    "Italy", "Brazil", "Canada", "Russia", "South Korea",
    "Australia", "Spain", "Mexico"),
  Year = rep(2023, 15),
  GDP_per_capita = c(69287, 12720, 33834, 52824, 2612, 46125, 44229, 36812,
    10412, 53247, 13817, 33121, 64491, 30115, 12670)
)
print(gdp_data)
png("plots/gdp.png", width = 720, height = 620)
par(mar = c(5, 8, 4, 2))
barplot(gdp_data$GDP_per_capita, names.arg = gdp_data$Country, horiz = TRUE, las = 1,
  col = "steelblue", main = "GDP per Capita by Country (2023)",
  xlab = "GDP per Capita (USD)", cex.names = 0.8)
dev.off()
```

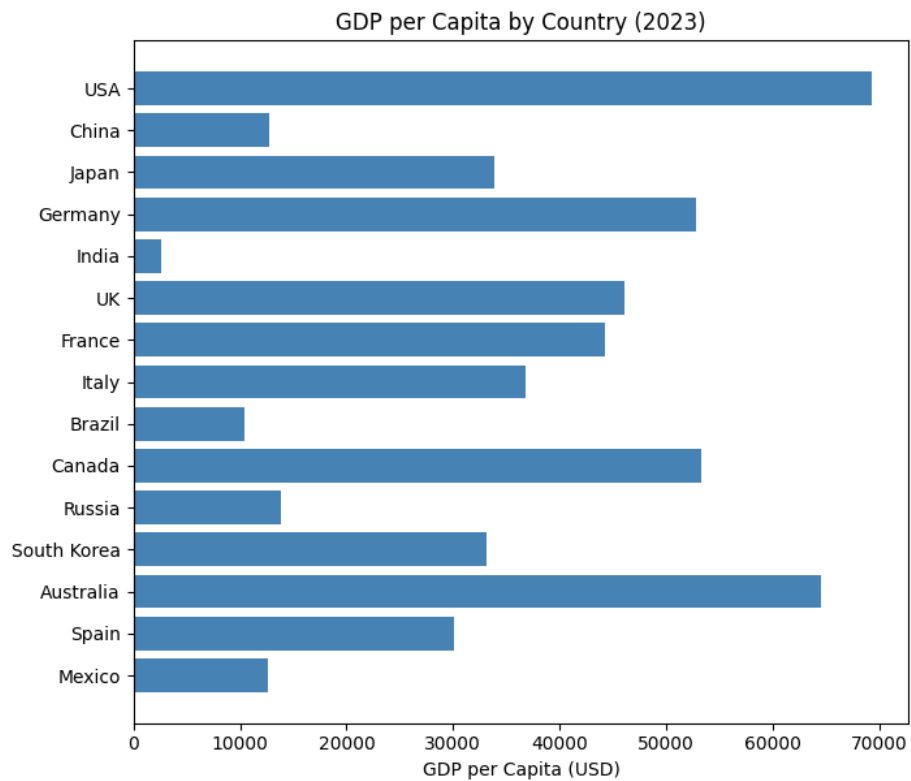
Output:

```
      Country Year GDP_per_capita
1      USA 2023      69287
```

```

2      China 2023      12720
3      Japan 2023      33834
4      Germany 2023    52824
5      India 2023      2612
6      UK 2023        46125
7      France 2023    44229
8      Italy 2023     36812
9      Brazil 2023    10412
10     Canada 2023    53247
11     Russia 2023    13817
12     South Korea 2023 33121
13     Australia 2023 64491
14     Spain 2023     30115
15     Mexico 2023    12670
null device
      1

```



GDP per capita by country (2023)

E. Write the script to print the content of a .csv file on the screen.

R Code:

```
write.csv(gdp_data, "sample.csv", row.names = FALSE)
csv_content <- read.csv("sample.csv")
cat("Content of sample.csv:\n")
print(csv_content)
```

Output:

Content of sample.csv:

	Country	Year	GDP_per_capita
1	USA	2023	69287
2	China	2023	12720
3	Japan	2023	33834
4	Germany	2023	52824
5	India	2023	2612
6	UK	2023	46125
7	France	2023	44229
8	Italy	2023	36812
9	Brazil	2023	10412
10	Canada	2023	53247
11	Russia	2023	13817
12	South Korea	2023	33121
13	Australia	2023	64491
14	Spain	2023	30115
15	Mexico	2023	12670

F. The data analyst at Argon Technology, Mr. John, needs the salaries of 10 employees in R (table of Sr.No, Name of employees, Salaries). (a) Which R command will Mr. John use to enter these values to demonstrate the output? (b) Now Mr. John wants to add the salaries of 5 new employees to the existing table in R. Demonstrate the output.

R Code:

```
Sr.No <- 1:10
Name.of.employees <- c("Vivek", "Karan", "James", "Soham", "Renu",
                       "Farah", "Hetali", "Mary", "Ganesh", "Krish")
Salaries <- c(21000, 55000, 67000, 50000, 54000, 40000, 30000, 70000, 20000, 15000)
emp_table <- data.frame(Sr.No, Name.of.employees, Salaries)
cat("F.a: The c() command (combine) is used along with data.frame() to enter\n")
cat("    the values and construct the table:\n")
print(emp_table)

cat("\nF.b: rbind() is used to add the 5 new employee records to the existing table:\n")
new_emp <- data.frame(Sr.No = 11:15,
                      Name.of.employees = c("Neha", "Amit", "Suresh", "Priya", "Rahul"),
                      Salaries = c(45000, 38000, 62000, 29000, 71000))
updated_emp_table <- rbind(emp_table, new_emp)
print(updated_emp_table)
```

Output:

F.a: The `c()` command (combine) is used along with `data.frame()` to enter the values and construct the table:

	Sr.No	Name.of.employees	Salaries
1	1	Vivek	21000
2	2	Karan	55000
3	3	James	67000
4	4	Soham	50000
5	5	Renu	54000
6	6	Farah	40000
7	7	Hetal	30000
8	8	Mary	70000
9	9	Ganesh	20000
10	10	Krish	15000

F.b: `rbind()` is used to add the 5 new employee records to the existing table:

	Sr.No	Name.of.employees	Salaries
1	1	Vivek	21000
2	2	Karan	55000
3	3	James	67000
4	4	Soham	50000
5	5	Renu	54000
6	6	Farah	40000
7	7	Hetal	30000
8	8	Mary	70000
9	9	Ganesh	20000
10	10	Krish	15000
11	11	Neha	45000
12	12	Amit	38000
13	13	Suresh	62000
14	14	Priya	29000
15	15	Rahul	71000

G. Consider the following data frame of subject, class and marks. (a) Create a subset of subjects less than 4 using the `subset()` function and demonstrate the output. (b) Create a subset where the subject column is less than 3 and the class column equals 2, and demonstrate the output (using `subset()` and, alternatively, using `df[]` brackets).

R Code:

```
subj_df <- data.frame(subject = c(1, 2, 3, 4, 5, 6),
                      class   = c(1, 2, 1, 2, 1, 2),
                      marks   = c(56, 75, 48, 69, 84, 53))

print(subj_df)
```

```

cat("\nG.a: subset() where subject < 4\n")
sub_a <- subset(subj_df, subject < 4)
print(sub_a)

cat("\nG.b: subset() where subject < 3 AND class == 2\n")
sub_b1 <- subset(subj_df, subject < 3 & class == 2)
print(sub_b1)

cat("\nG.b (alternative): using df[] brackets where subject < 3 AND class == 2\n")
sub_b2 <- subj_df[subj_df$subject < 3 & subj_df$class == 2, ]
print(sub_b2)

```

Output:

	subject	class	marks
1	1	1	56
2	2	2	75
3	3	1	48
4	4	2	69
5	5	1	84
6	6	2	53

G.a: subset() where subject < 4

	subject	class	marks
1	1	1	56
2	2	2	75
3	3	1	48

G.b: subset() where subject < 3 AND class == 2

	subject	class	marks
2	2	2	75

G.b (alternative): using df[] brackets where subject < 3 AND class == 2

	subject	class	marks
2	2	2	75

H. Write the script to sort the vector c(23,45,10,34,89,20,67,99) in ascending and descending order. Demonstrate the output.

R Code:

```

vec_h <- c(23, 45, 10, 34, 89, 20, 67, 99)
cat("Original vector:\n"); print(vec_h)
cat("Ascending order (sort):\n")
print(sort(vec_h))
cat("Descending order (sort with decreasing = TRUE):\n")
print(sort(vec_h, decreasing = TRUE))

```


Output:

```
Original vector:
[1] 23 45 10 34 89 20 67 99
Ascending order (sort):
[1] 10 20 23 34 45 67 89 99
Descending order (sort with decreasing = TRUE):
[1] 99 89 67 45 34 23 20 10
```

I. Create a list containing a numeric vector, a character vector, and a logical vector. Write the script to access each element of the list individually and demonstrate.

R Code:

```
my_list <- list(numbers = c(1, 2, 3, 4, 5),
               chars    = c("a", "b", "c"),
               logical  = c(TRUE, FALSE, TRUE))
cat("Full list:\n"); print(my_list)
cat("Numeric vector element (by name):\n"); print(my_list$numbers)
cat("Character vector element (by index [[2]]):\n"); print(my_list[[2]])
cat("Logical vector element (by name):\n"); print(my_list$logical)
```

Output:

```
Full list:
$numbers
[1] 1 2 3 4 5

$chars
[1] "a" "b" "c"

$logical
[1] TRUE FALSE TRUE

Numeric vector element (by name):
[1] 1 2 3 4 5
Character vector element (by index [[2]]):
[1] "a" "b" "c"
Logical vector element (by name):
[1] TRUE FALSE TRUE
```

J. For the vector of employee names `c("Amit","Sara","John","Meera","Ravi")`, write R code to: find the length of the vector, convert all names to uppercase, and sort the names alphabetically.

R Code:

```
emp_names <- c("Amit", "Sara", "John", "Meera", "Ravi")
cat("Length of vector:", length(emp_names), "\n")
upper_names <- toupper(emp_names)
cat("Uppercase names:\n"); print(upper_names)
sorted_names <- sort(emp_names)
cat("Alphabetically sorted names:\n"); print(sorted_names)
```

Output:

```
Length of vector: 5
Uppercase names:
[1] "AMIT" "SARA" "JOHN" "MEERA" "RAVI"
Alphabetically sorted names:
[1] "Amit" "John" "Meera" "Ravi" "Sara"
```

K. Write an R script to handle missing values (NA) in the vector `c(12,NA,45,NA,67,23)`. Demonstrate how to: (a) identify the missing values using `is.na()`, (b) remove the missing values, (c) replace the missing values with the mean of the remaining values.

R Code:

```
v_k <- c(12, NA, 45, NA, 67, 23)
cat("Original vector:\n"); print(v_k)
cat("K.a: Identify missing values using is.na()\n")
print(is.na(v_k))
cat("K.b: Remove missing values\n")
v_removed <- v_k[!is.na(v_k)]
print(v_removed)
cat("K.c: Replace missing values with the mean of remaining values\n")
v_replaced <- v_k
v_replaced[is.na(v_replaced)] <- mean(v_k, na.rm = TRUE)
print(v_replaced)
```

Output:

```
Original vector:
[1] 12 NA 45 NA 67 23
K.a: Identify missing values using is.na()
[1] FALSE TRUE FALSE TRUE FALSE FALSE
K.b: Remove missing values
[1] 12 45 67 23
K.c: Replace missing values with the mean of remaining values
[1] 12.00 36.75 45.00 36.75 67.00 23.00
```

L. Write R code to create a 3x3 matrix with values 1 to 9, and demonstrate how to access the second row and the third column separately.

R Code:

```
mat_1 <- matrix(1:9, nrow = 3, byrow = TRUE)
cat("3x3 matrix:\n"); print(mat_1)
cat("Second row:\n"); print(mat_1[2, ])
cat("Third column:\n"); print(mat_1[, 3])
```

Output:

```
3x3 matrix:
      [,1] [,2] [,3]
[1,]     1     2     3
[2,]     4     5     6
[3,]     7     8     9
Second row:
[1] 4 5 6
Third column:
[1] 3 6 9
```

M. Write an R script using a for loop to print the first 10 numbers of the Fibonacci series.

R Code:

```
fib <- numeric(10)
fib[1] <- 0
fib[2] <- 1
for (i in 3:10) {
  fib[i] <- fib[i - 1] + fib[i - 2]
}
cat("First 10 Fibonacci numbers:\n")
for (i in 1:10) cat(fib[i], " ")
cat("\n")
```

Output:

```
First 10 Fibonacci numbers:
0 1 1 2 3 5 8 13 21 34
```

N. Write an R function called `calculate_bmi` that takes weight (kg) and height (m) as arguments and calculates the BMI along with its category (Underweight, Normal, Overweight,

Obese). Demonstrate the output for at least 3 sample inputs.

R Code:

```
calculate_bmi <- function(weight, height) {
  bmi <- weight / (height^2)
  category <- if (bmi < 18.5) {
    "Underweight"
  } else if (bmi < 25) {
    "Normal"
  } else if (bmi < 30) {
    "Overweight"
  } else {
    "Obese"
  }
  cat(sprintf("Weight = %.1f kg, Height = %.2f m -> BMI = %.2f, Category = %s\n",
    weight, height, bmi, category))
  invisible(list(bmi = bmi, category = category))
}
calculate_bmi(50, 1.60)
calculate_bmi(70, 1.75)
calculate_bmi(95, 1.70)
```

Output:

```
Weight = 50.0 kg, Height = 1.60 m -> BMI = 19.53, Category = Normal
Weight = 70.0 kg, Height = 1.75 m -> BMI = 22.86, Category = Normal
Weight = 95.0 kg, Height = 1.70 m -> BMI = 32.87, Category = Obese
```

O. Consider the built-in dataset mtcars. Write R code to: (A) display the first 6 rows using head(), (B) filter cars with mpg greater than 20, (C) create a scatter plot of mpg vs hp.

R Code:

```
data(mtcars)
cat("0.A: First 6 rows using head()\n")
print(head(mtcars))
cat("0.B: Cars with mpg greater than 20\n")
print(mtcars[mtcars$mpg > 20, ])
png("plots/mtcars_scatter_0.png", width = 620, height = 420)
plot(mtcars$hp, mtcars$mpg, main = "MPG vs HP (mtcars)", xlab = "Horsepower",
  ylab = "Miles per Gallon", pch = 19, col = "darkblue")
dev.off()
cat("0.C: Scatter plot of mpg vs hp saved as plot.\n")
```

Output:

0.A: First 6 rows using head()

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

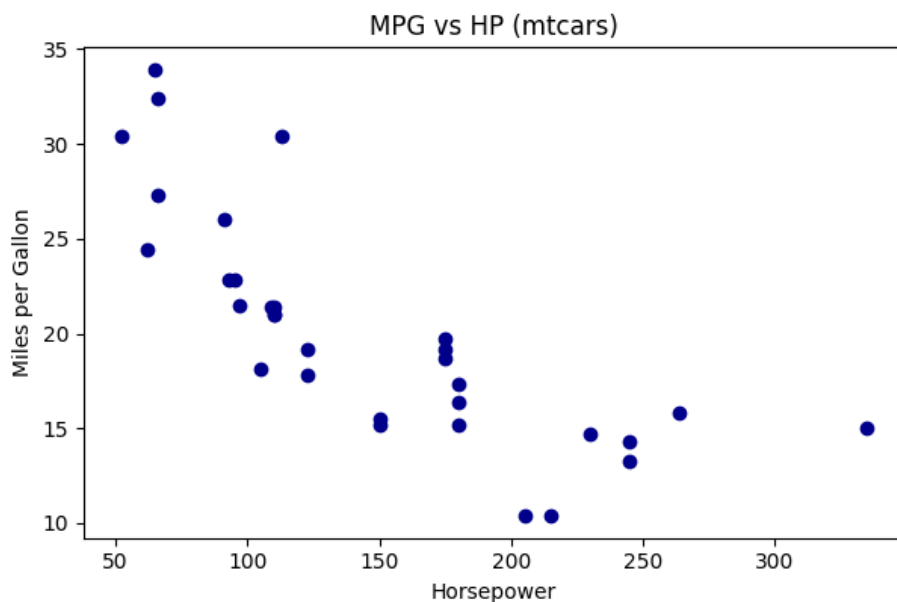
0.B: Cars with mpg greater than 20

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

null device

1

0.C: Scatter plot of mpg vs hp saved as plot.



Scatter plot of mpg vs hp (mtcars)

P. Explain the difference between `sapply()`, `lapply()` and `mapply()` with a suitable example for each, using the numeric vector `c(4,16,25,36,49)`.

R Code:

```
nums_p <- c(4, 16, 25, 36, 49)
cat("sapply(): returns a simplified vector\n")
print(sapply(nums_p, sqrt))
cat("lapply(): always returns a list\n")
print(lapply(nums_p, sqrt))
cat("mapply(): applies a function over multiple vectors in parallel\n")
print(mapply(function(x, y) x + y, nums_p, 1:5))
cat("Explanation: sapply() simplifies the result to a vector/matrix when\n")
cat("possible, lapply() always returns a list regardless of simplification,\n")
cat("and mapply() is the multivariate version that iterates over several\n")
cat("vectors/arguments simultaneously.\n")
```

Output:

```
sapply(): returns a simplified vector
[1] 2 4 5 6 7
lapply(): always returns a list
[[1]]
[1] 2

[[2]]
[1] 4

[[3]]
[1] 5

[[4]]
[1] 6

[[5]]
[1] 7

mapply(): applies a function over multiple vectors in parallel
[1] 5 18 28 40 54
Explanation: sapply() simplifies the result to a vector/matrix when
possible, lapply() always returns a list regardless of simplification,
and mapply() is the multivariate version that iterates over several
vectors/arguments simultaneously.
```

Q. Write an R script to read a .csv file containing student marks and demonstrate how to: (a) display the structure of the data using `str()`, (b) display summary statistics using `summary()`, (c) calculate the average marks of all students.

R Code:

```
marks_df <- data.frame(StudentID = 1:8,
                        Name = c("A", "B", "C", "D", "E", "F", "G", "H"),
                        Marks = c(56, 75, 48, 69, 84, 53, 90, 44))
write.csv(marks_df, "marks.csv", row.names = FALSE)
marks_read <- read.csv("marks.csv")
cat("Q.a: Structure of the data using str()\n")
str(marks_read)
cat("\nQ.b: Summary statistics using summary()\n")
print(summary(marks_read))
cat("\nQ.c: Average marks of all students\n")
cat(mean(marks_read$Marks), "\n")
```

Output:

```
Q.a: Structure of the data using str()
'data.frame': 8 obs. of 3 variables:
 $ StudentID: int 1 2 3 4 5 6 7 8
 $ Name      : chr "A" "B" "C" "D" ...
 $ Marks     : int 56 75 48 69 84 53 90 44
```

```
Q.b: Summary statistics using summary()
  StudentID      Name      Marks
Min.   :1.00  Length:8    Min.   :44.00
1st Qu.:2.75  Class :character 1st Qu.:51.75
Median :4.50  Mode  :character Median :62.50
Mean   :4.50                      Mean   :64.88
3rd Qu.:6.25                      3rd Qu.:77.25
Max.   :8.00                      Max.   :90.00
```

```
Q.c: Average marks of all students
64.875
```

R. Consider the built-in dataset `mtcars`. Write R code to: (a) display the first 6 rows using `head()`, (b) filter cars with `mpg` greater than 20, (c) create a scatter plot of `mpg` vs `hp`.

R Code:

```
cat("R.a: First 6 rows using head()\n")
print(head(mtcars))
```

```

cat("R.b: Cars with mpg greater than 20\n")
print(mtcars[mtcars$mpg > 20, ])
png("plots/mtcars_scatter_R.png", width = 620, height = 420)
plot(mtcars$hp, mtcars$mpg, main = "MPG vs HP (mtcars)", xlab = "Horsepower",
      ylab = "Miles per Gallon", pch = 19, col = "darkred")
dev.off()
cat("R.c: Scatter plot of mpg vs hp saved as plot.\n")

```

Output:

R.a: First 6 rows using head()

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225	105	2.76	3.460	20.22	1	0	3	1

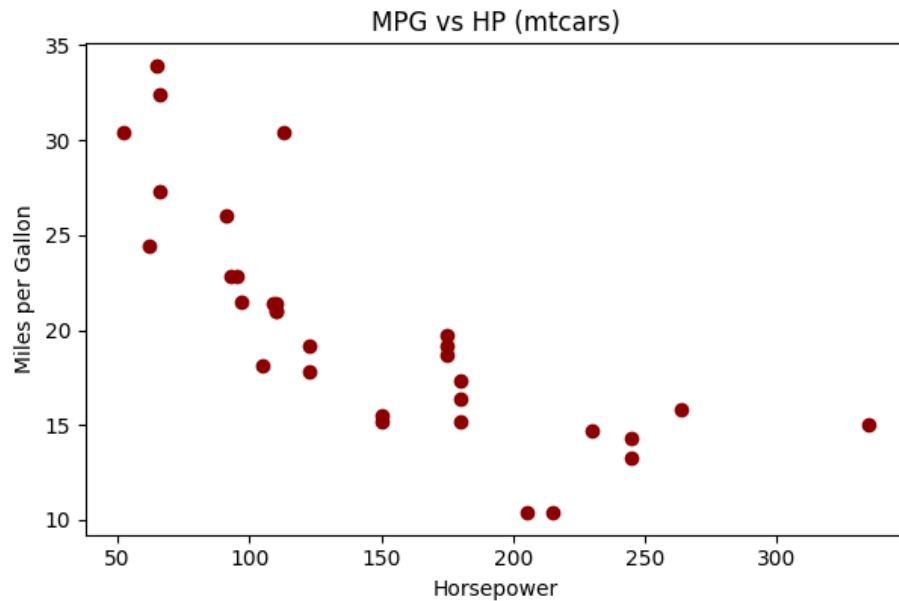
R.b: Cars with mpg greater than 20

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1
Porsche 914-2	26.0	4	120.3	91	4.43	2.140	16.70	0	1	5	2
Lotus Europa	30.4	4	95.1	113	3.77	1.513	16.90	1	1	5	2
Volvo 142E	21.4	4	121.0	109	4.11	2.780	18.60	1	1	4	2

null device

1

R.c: Scatter plot of mpg vs hp saved as plot.



Scatter plot of mpg vs hp (mtcars)

S. Write R code to create a factor variable for the following blood groups: c("A","B","O","AB","A","O","AB","B"). Demonstrate: (a) display the levels of the factor, (b) get the frequency count of each blood group using table().

R Code:

```
blood <- factor(c("A", "B", "O", "AB", "A", "O", "AB", "B"))
cat("S.a: Levels of the factor\n")
print(levels(blood))
cat("S.b: Frequency count of each blood group using table()\n")
print(table(blood))
```

Output:

```
S.a: Levels of the factor
[1] "A" "AB" "B" "O"
S.b: Frequency count of each blood group using table()
blood
  A AB  B  O
  2  2  2  2
```

T. Write a script to merge the following two data frames using the common column EmpID: df1 (EmpID, Name), df2 (EmpID, Department). Demonstrate the output using merge().

R Code:

```
df1 <- data.frame(EmpID = c(1, 2, 3, 4), Name = c("Amit", "Sara", "John", "Meera"))
df2t <- data.frame(EmpID = c(1, 2, 3, 4), Department = c("HR", "IT", "Finance", "Sales"))
cat("df1:\n"); print(df1)
cat("df2:\n"); print(df2t)
merged_df <- merge(df1, df2t, by = "EmpID")
cat("Merged data frame (by EmpID) using merge():\n")
print(merged_df)
```

Output:

```
df1:
  EmpID Name
1     1  Amit
2     2  Sara
3     3  John
4     4 Meera

df2:
  EmpID Department
1     1         HR
2     2         IT
3     3    Finance
4     4        Sales

Merged data frame (by EmpID) using merge():
  EmpID Name Department
1     1  Amit         HR
2     2  Sara         IT
3     3  John    Finance
4     4 Meera        Sales
```

U. Write R code to generate a sequence of dates from "2024-01-01" to "2024-01-10" and calculate the difference in days between the first and last date.

R Code:

```
date_seq <- seq(as.Date("2024-01-01"), as.Date("2024-01-10"), by = "day")
cat("Sequence of dates:\n"); print(date_seq)
diff_days <- difftime(date_seq[length(date_seq)], date_seq[1], units = "days")
cat("Difference in days between first and last date:", as.numeric(diff_days), "\n")
```

Output:

```
Sequence of dates:
[1] "2024-01-01" "2024-01-02" "2024-01-03" "2024-01-04" "2024-01-05" "2024-01-06"
[7] "2024-01-07" "2024-01-08" "2024-01-09" "2024-01-10"
Difference in days between first and last date: 9
```

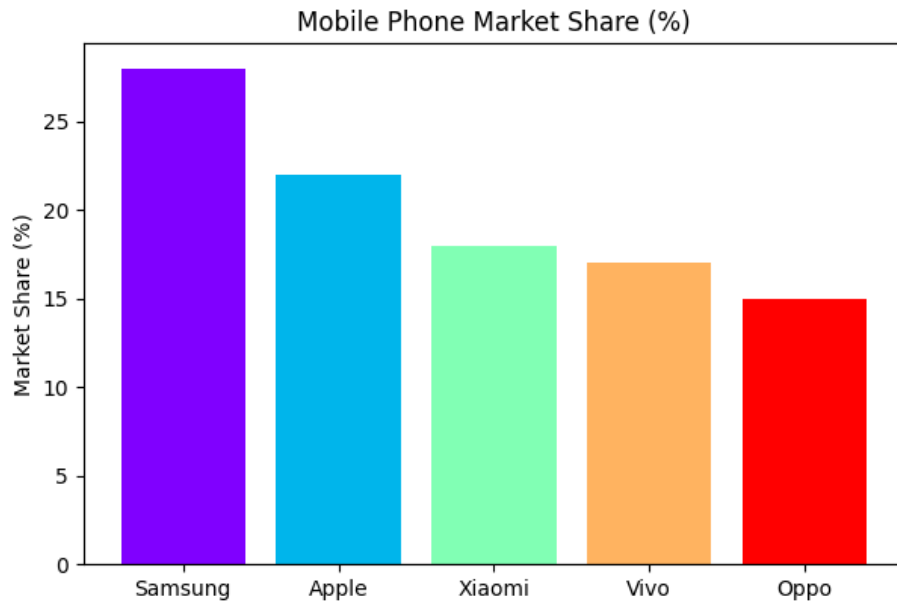
V. Create a bar chart and a pie chart in R to represent the market share (%) of 5 mobile phone brands. Use sample data of your choice and label them appropriately.

R Code:

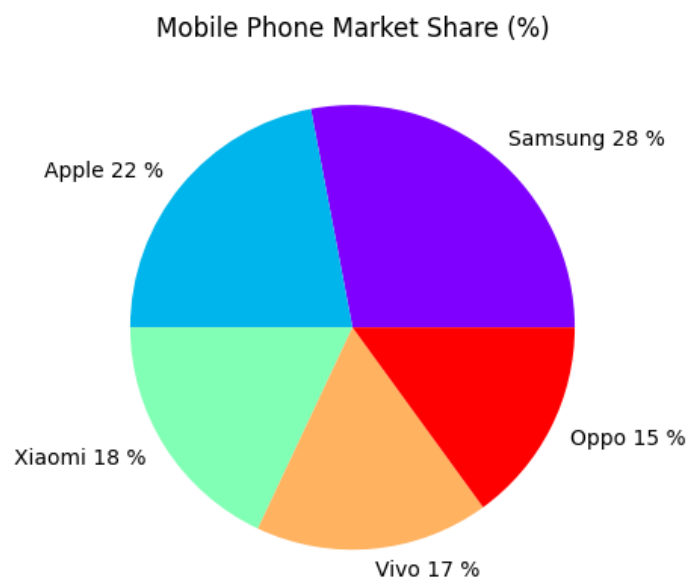
```
brands <- c("Samsung", "Apple", "Xiaomi", "Vivo", "Oppo")
market_share <- c(28, 22, 18, 17, 15)
mobile_df <- data.frame(Brand = brands, MarketShare = market_share)
print(mobile_df)
png("plots/bar_v.png", width = 620, height = 420)
barplot(market_share, names.arg = brands, col = rainbow(5),
        main = "Mobile Phone Market Share (%)", ylab = "Market Share (%)")
dev.off()
png("plots/pie_v.png", width = 620, height = 420)
pie(market_share, labels = paste(brands, market_share, "%"), col = rainbow(5),
    main = "Mobile Phone Market Share (%)")
dev.off()
```

Output:

```
      Brand MarketShare
1 Samsung           28
2  Apple           22
3  Xiaomi           18
4   Vivo           17
5   Oppo           15
null device
      1
null device
      1
```



Mobile phone market share - bar chart



Mobile phone market share - pie chart

W. Write a script to create a simple linear regression model using two numeric variables from the mtcars dataset (wt and mpg). Print the model summary and plot the regression line.

R Code:

```
model_w <- lm(mpg ~ wt, data = mtcars)
```

```
print(summary(model_w))
png("plots/regression_w.png", width = 620, height = 420)
plot(mtcars$wt, mtcars$mpg, main = "Linear Regression: MPG vs WT", xlab = "Weight (1000 lbs)",
     ylab = "Miles per Gallon", pch = 19, col = "darkgreen")
abline(model_w, col = "red", lwd = 2)
dev.off()
```

Output:

Call:

```
lm(formula = mpg ~ wt, data = mtcars)
```

Residuals:

Min	1Q	Median	3Q	Max
-4.5432	-2.3647	-0.1252	1.4096	6.8727

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	37.2851	1.8776	19.858	< 2e-16 ***
wt	-5.3445	0.5591	-9.559	1.29e-10 ***

 Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

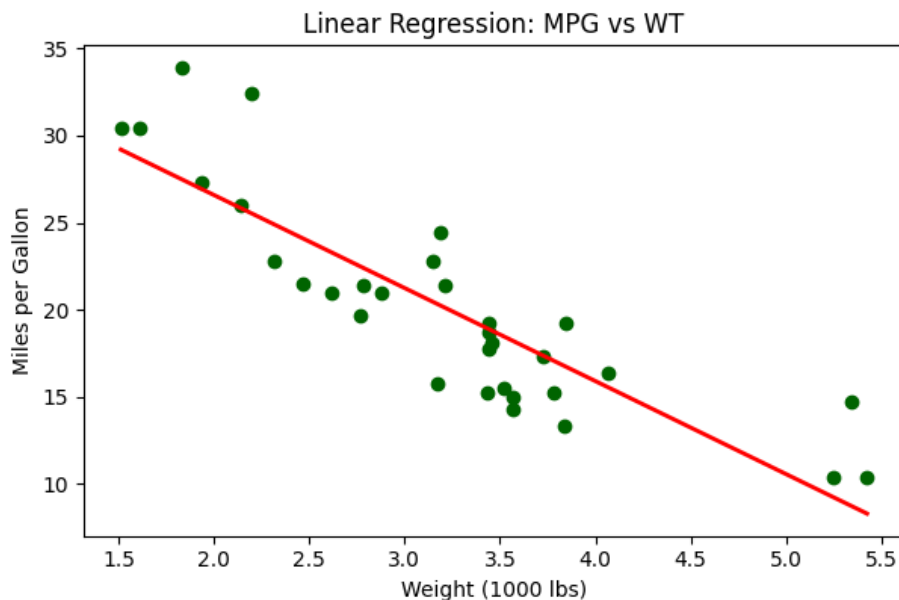
Residual standard error: 3.046 on 30 degrees of freedom

Multiple R-squared: 0.7528, Adjusted R-squared: 0.7446

F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10

null device

1



Linear regression: mpg vs wt with fitted line

X. Write R code to demonstrate the use of `ifelse()` to classify a vector of exam scores `c(45,78,92,33,61,85)` as "Pass" (score ≥ 40) or "Fail" (score < 40).

R Code:

```
scores <- c(45, 78, 92, 33, 61, 85)
result_x <- ifelse(scores >= 40, "Pass", "Fail")
print(data.frame(Score = scores, Result = result_x))
```

Output:

	Score	Result
1	45	Pass
2	78	Pass
3	92	Pass
4	33	Fail
5	61	Pass
6	85	Pass

Y. Write an R script to create a data frame of 5 products with columns Product, Price and Quantity. Add a new column Total that calculates Price x Quantity. Demonstrate the output.

R Code:

```
products <- data.frame(Product = c("Pen", "Notebook", "Bag", "Bottle", "Umbrella"),
                        Price = c(10, 50, 800, 150, 300),
                        Quantity = c(100, 60, 15, 40, 20))
products$Total <- products$Price * products$Quantity
print(products)
```

Output:

	Product	Price	Quantity	Total
1	Pen	10	100	1000
2	Notebook	50	60	3000
3	Bag	800	15	12000
4	Bottle	150	40	6000
5	Umbrella	300	20	6000

Z. Explain and demonstrate with code the difference between `rbind()` and `cbind()` using two sample vectors.

R Code:

```
v1 <- c(1, 2, 3)
v2 <- c(4, 5, 6)
cat("Vector v1:\n"); print(v1)
cat("Vector v2:\n"); print(v2)
cat("rbind(v1, v2) - combines as rows (stacked vertically):\n")
print(rbind(v1, v2))
cat("cbind(v1, v2) - combines as columns (placed side by side):\n")
print(cbind(v1, v2))
cat("Explanation: rbind() binds vectors/data frames row-wise (each vector\n")
cat("becomes a row of the resulting matrix), while cbind() binds them\n")
cat("column-wise (each vector becomes a column).\n")
```

Output:

```
Vector v1:
[1] 1 2 3
Vector v2:
[1] 4 5 6
rbind(v1, v2) - combines as rows (stacked vertically):
  [,1] [,2] [,3]
v1    1    2    3
v2    4    5    6
cbind(v1, v2) - combines as columns (placed side by side):
      v1 v2
[1,]  1  4
[2,]  2  5
[3,]  3  6
Explanation: rbind() binds vectors/data frames row-wise (each vector
becomes a row of the resulting matrix), while cbind() binds them
column-wise (each vector becomes a column).

===== END OF SCRIPT =====
```